

Architecture Reference for an Edge-Based RF Spectrum Sensing Node

A Conceptual and Methodological Description of a
Raspberry Pi 5 / HackRF One / GPS-LTE / RF-Multiplexer Sensor

Prepared for internal research review

July 3, 2026

Abstract

This document describes, at the conceptual and methodological level, the architecture of an edge-deployed radio-frequency (RF) spectrum sensing node built around a Raspberry Pi 5 host, a HackRF One software-defined radio (SDR) front end, an RF multiplexer for antenna-port selection, and a GPS/LTE module for geolocation, time reference, and backhaul connectivity. The system is designed as a dual-plane architecture: a real-time *data plane* implemented in C, responsible for IQ acquisition and digital signal processing (DSP) under bounded latency, and a supervisory *control plane* implemented in Python, responsible for scheduling, telemetry, and communication with a remote coordination server. We describe the design principles that govern the separation of these planes, the methodologies used for spectral estimation, calibration, fault recovery, and data integrity, and we position the system relative to existing state-of-the-art (SOTA) distributed spectrum-sensing platforms. Where a subsystem is known to exist and to be operationally load-bearing but is not documented in the source material available to the authors of this reference — specifically the internal design of the GPS/LTE engine and the RF multiplexer — we describe only its externally observable interface contract and explicitly flag the internal mechanism as undocumented, rather than inferring implementation detail that cannot be verified.

Contents

1	Introduction and Context	1
1.1	Motivation	1
1.2	Related Work and State of the Art	2
1.3	Scope of This Document	2
2	System Overview	2
3	Design Methodology and Principles	3
3.1	Data-Plane / Control-Plane Separation	3
3.2	Hardware Arbitration via a Global State Machine	3
3.3	Request-Scoped Acquisition Semantics	3
3.4	Fault Containment and Self-Healing	3
3.5	Persistence and Atomicity	4
4	Subsystem Architecture	4
4.1	RF Data Plane	4
4.2	Control Plane / Orchestration	5
4.3	GPS / LTE and RF Multiplexer Subsystem	6
5	Data Flow Methodology	7
6	Concurrency and State Model	8

7 Reliability and Fault-Tolerance Methodology	8
8 Comparative Positioning Relative to the State of the Art	9
9 Limitations and Open Questions	10
10 Conclusion	10

1 Introduction and Context

1.1 Motivation

Wide-area radio spectrum monitoring has traditionally relied on a small number of expensive, professionally operated monitoring stations. Regulatory bodies responsible for spectrum management — allocating frequency bands, detecting interference, and enforcing licensed use — have historically been constrained by the cost and limited geographic density of such stations. Over the last decade, the falling cost of software-defined radio front ends and single-board computers has made it feasible to deploy larger numbers of lower-cost, unattended sensing nodes at the network edge, trading some per-node measurement fidelity for spatial density, deployment flexibility, and continuous operation.

The system described here follows this edge-sensing paradigm. It is designed to run unattended for extended periods, to report both scheduled (“campaign”) and on-demand (“realtime”) spectral measurements to a remote coordination server, and to do so within the power, thermal, and compute envelope of a single-board computer.

1.2 Related Work and State of the Art

Distributed, low-cost spectrum sensing has an established research and deployment lineage. Platforms such as **ElectroSense** demonstrated that a Raspberry Pi paired with a low-cost SDR front end (originally RTL-SDR-class hardware, later extended with up/down-converters to reach into the low-GHz range) could be crowdsourced into a distributed sensing network with a shared backend and open API, performing on-device FFT-based feature extraction before streaming measurements to a cloud backend. Its successor, **ElectroSense+**, moved further processing to the edge and added encrypted remote-access and user-controlled demodulation services, illustrating a broader trend in the field: as edge compute capacity grows, more of the DSP burden migrates from the cloud back to the sensor itself, reducing backhaul bandwidth and enabling lower-latency local decisions (e.g., DC-spike removal, channel filtering, or modulation-quality estimation) before data ever leaves the node. Related distributed-sensing efforts (e.g., SpecSense-class systems, and IoT-oriented spectrum cartography research) share a common architectural skeleton: an inexpensive embedded compute platform, an SDR front end, a local acquisition/DSP stage, and a scheduling/reporting layer that talks to a central server.

The system described in this reference sits within that same lineage but differs from the canonical crowdsourced model in three respects that are methodologically significant:

- 1. Regulatory rather than crowdsourced operation.** The node reports to a single authoritative coordination server rather than a public crowdsourcing backend, and supports scheduled regulatory “campaigns” with defined time windows in addition to ad-hoc realtime queries.
- 2. A higher-performance, wider-range SDR front end.** HackRF One provides continuous tuning from below 1 MHz to 6 GHz at up to 20 Msps, removing the need for an auxiliary up/down-converter that RTL-SDR-class front ends typically require to reach the same range.

3. **A hard split between a real-time data plane and a supervisory control plane**, implemented in different languages and processes, communicating over a local IPC channel rather than being organized as a single monolithic acquisition process. This is discussed at length in Section 3.

1.3 Scope of This Document

This is a conceptual and methodological reference. It does not reproduce source code, function signatures, or file-level implementation detail; it describes *what the system does, why it is structured the way it is, and which design patterns are used to satisfy which requirements*. Where a component's internal design is not available in the documentation this reference was derived from, that gap is stated explicitly rather than filled by inference.

2 System Overview

The sensor node integrates four physical subsystems and two logical software planes on a single embedded host. Figure 1 shows the system as a vertical stack, from the remote coordination server down to the antenna.

The host operating system schedules the two software planes as independent, restart-supervised processes. This separation is the central methodological choice of the system and is discussed next.

3 Design Methodology and Principles

Five principles recur throughout the system and can be treated as the design methodology against which individual components should be evaluated.

3.1 Data-Plane / Control-Plane Separation

The system draws a hard boundary between code that must meet real-time deadlines (sample ingestion, DSP, hardware register access) and code that governs longer-horizon decisions (what to measure, when, and where to send it). The former is implemented in C, compiled to a native binary, and given real-time-adjacent scheduling considerations (thread affinity avoidance, passive wait policies). The latter is implemented in Python as a set of independently restartable services. The two communicate over a single, narrow, well-defined local IPC contract (request/reply, JSON-encoded) rather than sharing memory or process space.

The methodological benefit of this split is fault isolation: a defect or crash in the orchestration logic (scheduling, HTTP retry policy, telemetry formatting) cannot corrupt or stall the acquisition hot path, and a hardware-level fault in the SDR driver cannot bring down the process responsible for talking to the coordination server. Each plane can also be restarted independently by the service supervisor without requiring the other to reinitialize.

3.2 Hardware Arbitration via a Global State Machine

Because the SDR front end is a single physical resource, the system enforces mutual exclusion over acquisition activity at the control-plane level through a global state machine with a small number of mutually exclusive states (idle, on-demand/realtime acquisition, scheduled campaign acquisition, calibration). This is a coarse-grained but effective concurrency control mechanism: rather than relying on low-level locking around the hardware driver, the system prevents conflicting acquisition intents from ever being issued in the first place. Section 6 describes this model in more detail.

3.3 Request-Scoped Acquisition Semantics

Every acquisition is scoped to the request that triggered it. Before capturing samples for a given request, any data buffered prior to that request is discarded, so that the response is guaranteed to reflect spectrum conditions *after* the request was issued rather than stale, pre-existing buffer contents. This is a subtle but important correctness property for a system whose clients (regulatory operators, scheduled campaigns) expect a measurement to correspond to a specific point in time, not to whatever happened to be sitting in a buffer.

3.4 Fault Containment and Self-Healing

The system is designed on the assumption that most of its failure modes are transient: USB/SDR disconnects, IPC socket desynchronization, network timeouts, and thermal or power fluctuations typical of unattended field deployment. The methodology favors automatic, bounded-retry self-healing over operator intervention:

- IPC channels that enter an inconsistent state (e.g., a stalled request/reply handshake) are torn down and recreated rather than nursed back to a known state.
- Hardware sessions are health-checked before use and invalidated/reopened on failure, with a bounded number of recovery attempts.
- Long-running worker threads and processes check a cancellation flag on a short polling interval so that shutdown and recovery remain responsive even mid-operation.
- Process-level supervision (restart-always policies with lock-guarded singleton enforcement) forms the outermost layer of defense against unrecoverable in-process failures.

3.5 Persistence and Atomicity

State that must survive process restarts or be shared between the data plane and the control plane (e.g., calibration results, GPS fix, campaign lock flags) is written to a shared, memory-backed store using an atomic write discipline: write to a temporary location, flush, then atomically replace the target, under an advisory lock that distinguishes shared (read) from exclusive (write) access. This avoids partial-write corruption if the node loses power or a process is killed mid-write, which is a realistic occurrence for unattended, field-powered equipment.

4 Subsystem Architecture

4.1 RF Data Plane

The data plane's job is to turn a stream of raw IQ samples into either a spectral estimate or a demodulated audio stream, within bounded latency, on a single-board computer's compute budget. Conceptually it is organized as a pipeline (Figure 2):

1. **Ingest.** Samples are handed off from the SDR driver's callback context into a lock-free, single-producer/single-consumer circular buffer. This decouples the high-frequency hardware callback from the consumer(s) reading from it, so that neither can block the other.
2. **Signal conditioning.** Raw samples are converted to a normalized complex representation and corrected in place for DC offset, and I/Q gain and phase imbalance — endemic front-end impairments in direct-conversion SDR hardware that would otherwise distort spectral estimates.
3. **Optional channel isolation.** When a request specifies a sub-band of interest, a frequency-domain filter isolates it prior to further processing, using a two-stage approach: an outlier-

clipping stage to prevent strong out-of-band interferers from dominating the result, followed by a smooth-transition (raised-cosine) masking stage chosen specifically to minimize time-domain ringing artifacts.

4. **Spectral estimation or demodulation.** Depending on the request, the conditioned signal is either passed through a power spectral density (PSD) estimator (two interchangeable methods are supported, trading estimation quality against computational cost) or through an FM/AM demodulation chain that also derives simple modulation-quality metrics (deviation, depth) as a byproduct.
5. **Publish.** Results are serialized and returned over the same request/reply channel the request arrived on.

A concurrent, independently running audio path drains a secondary buffer continuously (rather than per-request) whenever demodulation is active, encodes PCM audio into a compressed frame format, and streams it out over a separate network channel — allowing a human operator to listen to a demodulated signal while spectral requests continue to be served on the primary path.

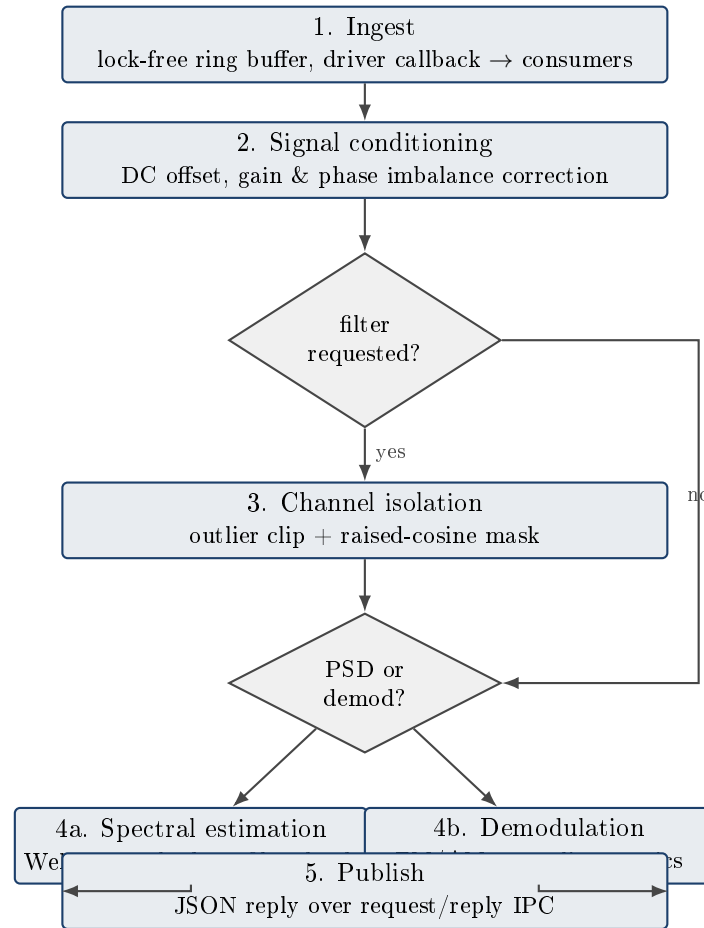


Figure 2: Conceptual DSP pipeline of the data plane, executed per acquisition request.

Calibration as a distinct methodology. Frequency reference error (PPM offset) is determined not by a fixed factory value but by an on-device, three-stage search: a wideband sweep identifies candidate strong signals, a pilot-tone detection stage (searching for the stereo pilot characteristic of FM broadcast) identifies which candidate is a usable reference, and a symmetry-based fine search around that reference estimates the frequency offset that minimizes sideband asymmetry. The result is smoothed across runs and bounded by sanity limits before being accepted,

which protects the system against a single noisy measurement corrupting the working calibration value. This is a self-calibrating methodology appropriate for a device that cannot assume factory calibration remains valid over years of field deployment and temperature variation.

4.2 Control Plane / Orchestration

The control plane is organized as a small set of independently scheduled responsibilities rather than one monolithic loop:

- **Realtime orchestration** polls the coordination server for on-demand configuration and relays it to the data plane, subject to the global state machine.
- **Campaign scheduling** treats longer-running, time-boxed measurement campaigns as a distinct concern from realtime polling: campaign definitions are fetched, persisted, and materialized into OS-level scheduled jobs, decoupling “deciding what should run and when” from “actually running it.”
- **Status reporting** runs on a fixed timer, independent of acquisition activity, collecting host health telemetry (CPU, memory, disk, temperature, network reachability) and reporting it regardless of whether the sensor is currently acquiring.
- **Retry queueing** decouples successful local measurement from successful remote delivery: a measurement that cannot be uploaded (server unreachable, network outage) is persisted locally and retried on a separate schedule, rather than being lost or blocking the acquisition path.

This decomposition follows a general reliability methodology for unattended field devices: each concern that can fail independently (acquisition, scheduling, reporting, delivery) is given its own failure domain and its own recovery schedule, so that a fault in one does not propagate into the others.

4.3 GPS / LTE and RF Multiplexer Subsystem

The internal design of the GPS/LTE module and the RF multiplexer is not documented in the material this reference is derived from. Both are known to be present and operationally used in the deployed system, but this reference deliberately does not speculate about their internal implementation. What can be stated with confidence is their *externally observable interface contract*, inferred from the data the rest of the system consumes and produces around them:

- A GPS fix (latitude, longitude, altitude, and an associated timestamp) is available to the control plane and is reported upstream to the coordination server as a distinct telemetry type, separate from spectral data and status reporting.
- An antenna-port selector, constrained to a small fixed range of physical ports, is part of the realtime and campaign configuration contract, implying that RF-path selection (i.e., which physical antenna feeds the SDR for a given request) is externally configurable per request rather than fixed at deployment time.
- Time/location data of this kind is conventionally also used, in comparable systems, as an input to timestamp discipline and PPM-drift bookkeeping; whether this system uses it in that way, versus purely for geotagging, is **not established** by the available documentation and should not be assumed.

Figure 3 represents this honestly: the subsystem is drawn as a boundary with a known contract and an unknown interior.

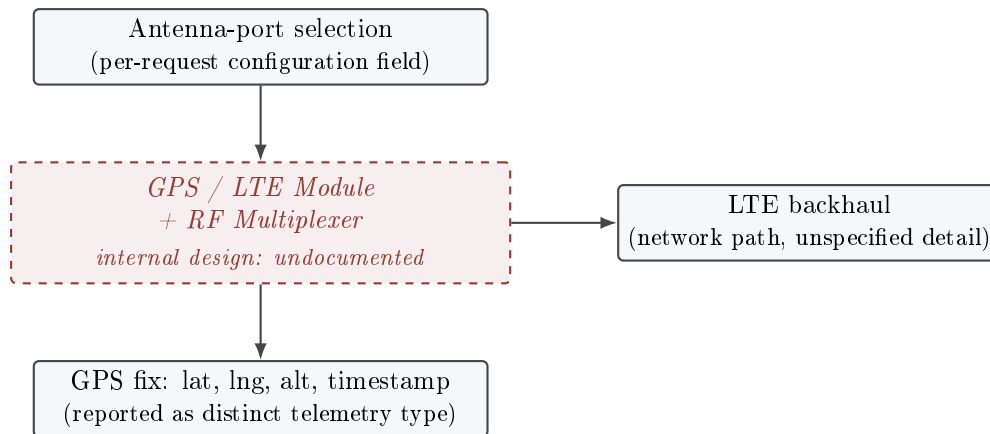


Figure 3: Interface-level view of the GPS/LTE/multiplexer subsystem. Only the boundary contract is known; the interior is explicitly out of scope for this reference.

5 Data Flow Methodology

Three data flows dominate normal operation. Each is shown as a vertical sequence in Figures 4–6.

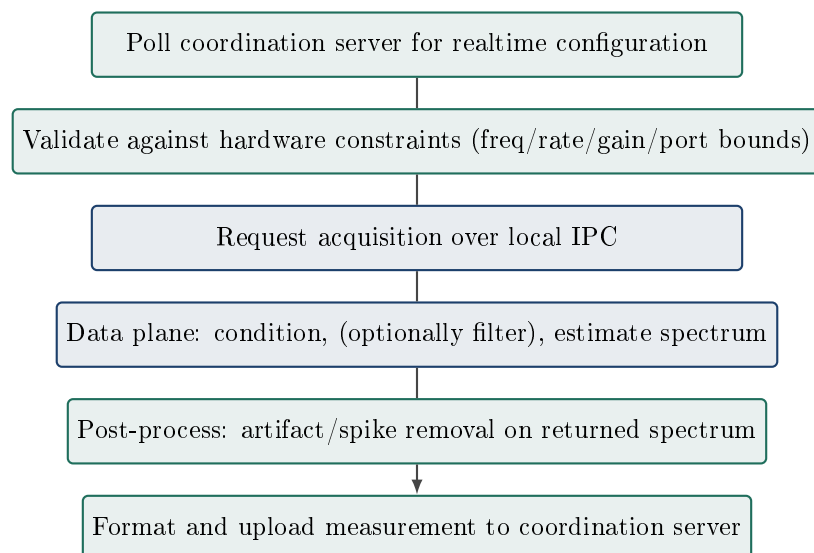


Figure 4: Realtime acquisition flow.

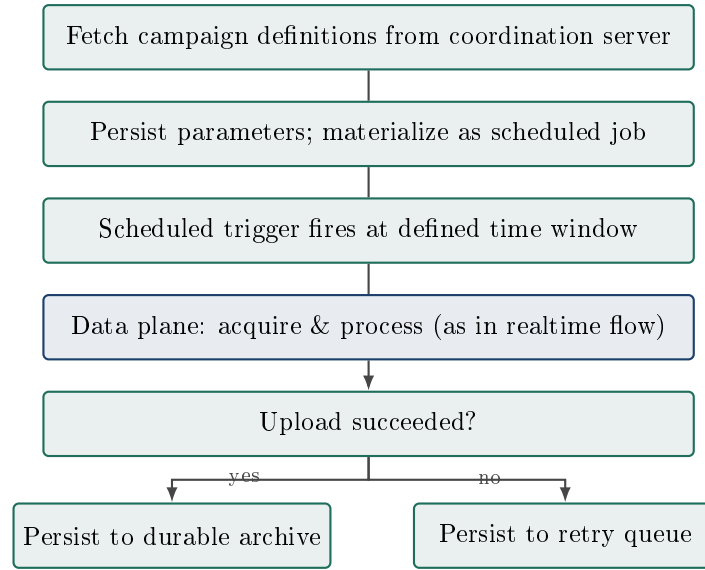


Figure 5: Scheduled campaign acquisition flow.

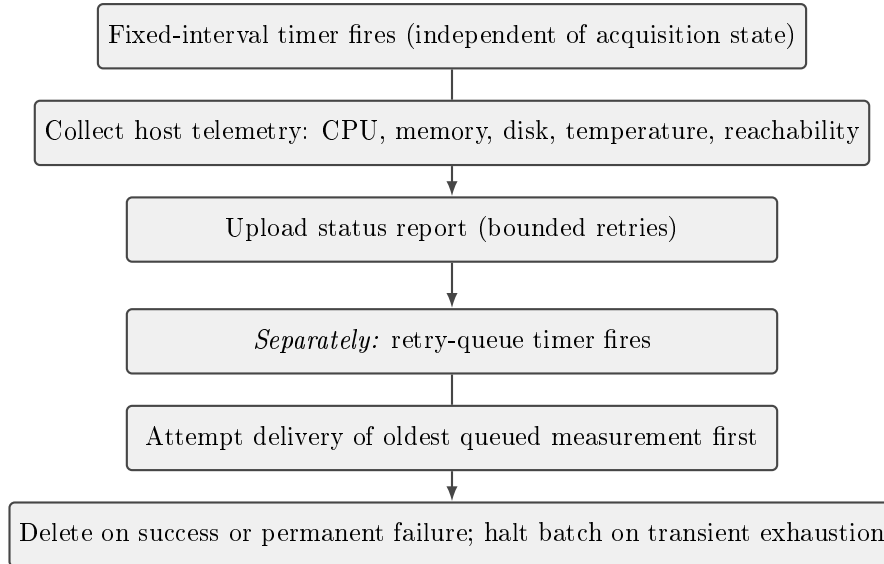


Figure 6: Status reporting and retry-queue flow (both timer-driven, independent of acquisition).

6 Concurrency and State Model

Acquisition-related concurrency is governed by a single global state machine rather than fine-grained locking. This is a deliberate simplification: because the underlying hardware resource (the SDR) is strictly single-owner, coarse mutual exclusion at the level of *intent* (“what kind of activity is the node currently committed to”) is sufficient and easier to reason about than lock-based coordination deep inside the acquisition path.

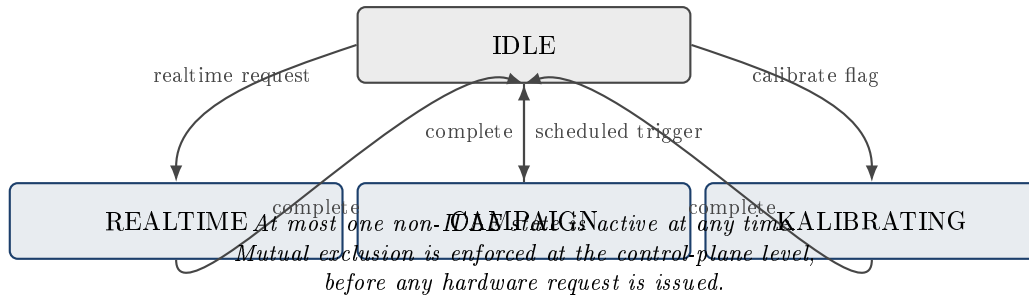


Figure 7: Global acquisition state machine. Transitions out of IDLE are gated by whichever trigger arrives first; the state machine itself is the mechanism preventing two acquisition intents from being active simultaneously.

7 Reliability and Fault-Tolerance Methodology

Table 1 summarizes the failure modes the architecture explicitly anticipates and the corresponding design pattern used to contain each one. This mapping is useful as a checklist when evaluating whether a proposed change preserves the system’s reliability posture.

Failure mode	Design pattern applied	Layer
SDR/USB disconnect or register-level fault	Session health check before each use; bounded-attempt device reopen	Data plane
Stalled IPC request/reply handshake	Destroy and recreate the socket, resetting protocol state	Both planes (paired)
Stale samples in acquisition buffer	Discard-to-head before each request (request-scoped capture)	Data plane
Coordination-server unreachable	Persist-and-retry queue, independent schedule from acquisition	Control plane
Process crash / unresponsive process	Restart-always service supervision with singleton locking	OS / service layer
Power loss mid-write to shared state	Write-temp → fsync → atomic replace, under advisory lock	Both planes (paired)
CPU/thermal overload from rapid requests	Configurable inter-request pacing (“cooldown”)	Data plane
Idle hardware power/thermal draw	Lazy hardware open; idle timeout closes device	Data plane
Concurrent conflicting acquisition intents	Global mutually-exclusive state machine	Control plane
Blocked thread delaying shutdown	Cooperative cancellation checked on short polling interval	Both planes
Frequency reference drift over time/temperature	On-device self-calibration with outlier-bounded smoothing	Data plane

Table 1: Failure modes anticipated by the architecture and the corresponding mitigation pattern.

8 Comparative Positioning Relative to the State of the Art

Table 2 situates the system, at a conceptual level, against the general class of distributed low-cost spectrum-sensing platforms exemplified by ElectroSense and its successors, without claiming precise, sourced performance figures for either side.

Dimension	Canonical crowdsourced platforms (e.g., ElectroSense-class)	This system
Deployment model	Open, crowdsourced; volunteer-hosted nodes	Regulator-operated; scheduled campaigns
SDR front end	RTL-SDR-class, often with an auxiliary up/down-converter for multi-GHz reach	HackRF One, native continuous coverage into the low-GHz range
Edge DSP scope	Evolved from cloud-side FFT toward increasing on-device processing (successor generations)	On-device compensation, filtering, PSD estimation, and demodulation from the outset
Backend interaction	Public/open API, crowdsourced backend	Single authoritative coordination server, private API
Acquisition modes	Primarily continuous/streaming sensing	Explicit dual mode: on-demand realtime <i>and</i> scheduled, time-boxed campaigns
Ancillary sensing	GPS commonly used for time sync	GPS/LTE module present; exact internal role not established (Section 4.3)

Table 2: Conceptual positioning against the general class of distributed spectrum-sensing platforms.

The broader field trend — pushing DSP work from a centralized backend toward the sensing edge as embedded compute capacity grows — is directly reflected in this system’s decision to perform IQ compensation, filtering, spectral estimation, artifact removal, and even self-calibration entirely on-node, with the backend receiving already-processed spectral and telemetry data rather than raw samples.

9 Limitations and Open Questions

- **GPS/LTE and multiplexer internals are undocumented.** As stated in Section 4.3, only the interface contract is known. A complete architecture reference would require either the source documentation for this subsystem or direct inspection of its implementation.
- **Single point of hardware failure.** The architecture assumes exactly one SDR front end; there is no redundant RF path, so a permanent hardware fault removes the node from service until physically serviced.
- **Permissive configuration parsing.** The control-plane/data-plane configuration contract is deliberately permissive (unrecognized fields ignored, malformed input falls back to defaults) to tolerate an imperfect network link. This is a reasonable trade-off for availability but means a malformed campaign definition could silently run with unintended parameters rather than failing loudly; this may warrant an explicit validation-failure signal in future iterations.
- **Authentication and transport security** of the remote API were out of scope for the documentation this reference was derived from and are not characterized here.

10 Conclusion

The sensor node’s architecture follows an edge-first methodology consistent with the broader trajectory of distributed spectrum-sensing research: minimize backhaul dependence by performing signal conditioning, spectral estimation, artifact removal, and self-calibration on-device; isolate the real-time acquisition path from supervisory logic through a strict data-plane/control-plane split; and favor coarse-grained, easily-reasoned-about concurrency control (a global state machine, request-scoped capture, atomic persistence) over fine-grained locking. The result is a node designed to operate unattended, recover automatically from the transient failure modes typical of field deployment, and support both on-demand and scheduled regulatory measurement workflows. The principal open gap in this reference is the internal design of the GPS/LTE and RF-multiplexer subsystem, which should be the subject of a follow-up documentation pass once its implementation is available for review.

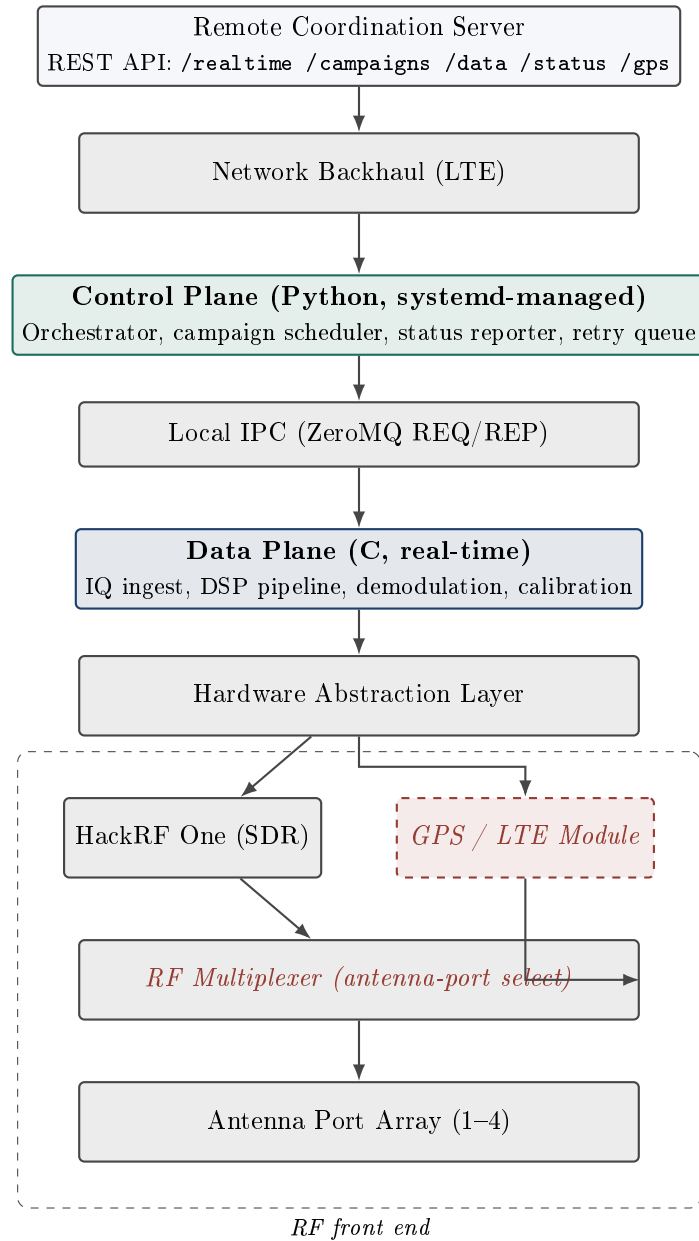


Figure 1: Vertical system stack. Solid blue/green boxes are documented in detail; dashed red boxes (GPS/LTE module, RF multiplexer) are known to exist and to be operationally used, but their internal design is not covered by the available documentation — only their externally observable interface contract is known (see Section 4.3).